PORTALDOT HACKER HOUSE 2026 · MODULAR CLOUD & MIDDLEWARE

Portal Meter

Pay-per-request API settlement on Portaldot V3,
built on both of its contract VMs.

github.com/swiftbotlab/portal-meter · swiftbotlab.github.io/portal-meter/demo

The problem

- APIs are now bought by **programs**: apps, bots, AI agents, indexers. They want to pay **per call**, cents or less.
- API keys + monthly plans don't fit a client that needs 40 calls today and none tomorrow.
- "Pay first" protocols (HTTP 402 / x402) move money but give the buyer **no recourse** when the call fails.
- And no way to know, before paying, whether a provider actually delivers.

What a machine buyer needs
price up front · pay only for what is served · automatic refund when it isn't · a public track record of the seller

Portal Meter

1. **GET** → `402` with a quote (price, token, endpoint, nonce)
2. Client pays **one request** into escrow: `MeterSettlement.pay()` on Portaldot's EVM; the request id is bound to the payer on-chain
3. Client retries with a signed proof → served **once**, with a provider-signed receipt
4. Provider **settles** (funds to payee) or **refunds**; the buyer can reclaim after a delay if the provider does nothing
5. The outcome is **anchored in an ink! registry** in the same transaction; buyers confirm or dispute
6. Anyone verifies a receipt: **ACCEPT / REJECT** with every check (CLI and browser)

Why Portaldot is essential

Native assets are ERC-20s

Every `pallet-assets` token is reachable from Solidity at a precompile address. Endpoints are priced in a native asset; no wrapped token to deploy or trust.

Two VMs, one identity

Escrow in Solidity (pallet-revive), provider registry in ink! (pallet-contracts). Both sides key accounts by the same EVM address, using the runtime's own mapping rule.

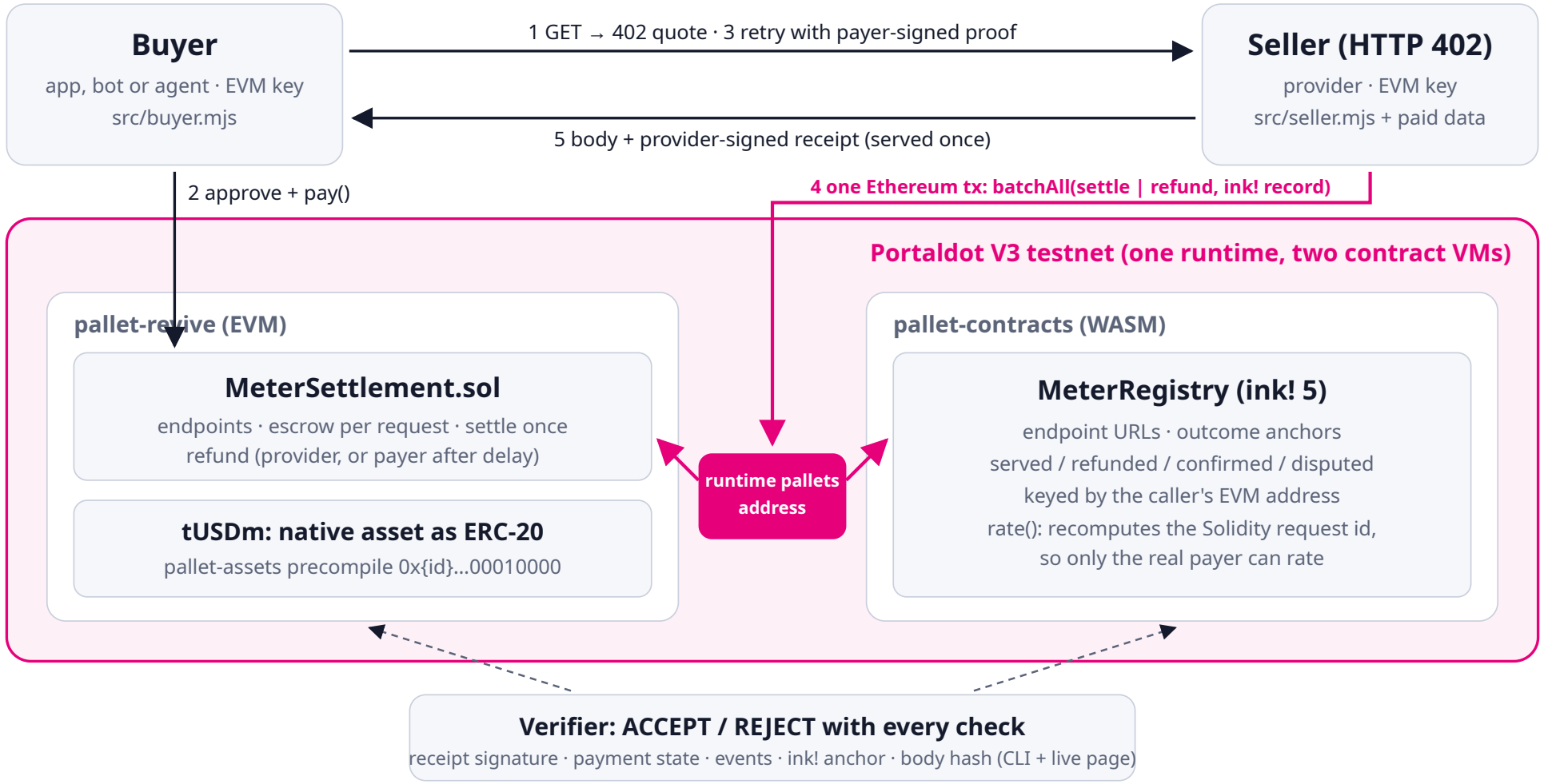
One transaction across both VMs

An Ethereum transaction to the runtime pallets address carries `utility.batchAll([revive.call(settle), contracts.call(record)])`: settlement and anchor succeed or fail together. A single-VM chain cannot do this.

Ink! checks EVM payers natively

The registry recomputes the Solidity request id (keccak + ABI) from the caller's address, so only the real payer can rate a request, with no bridge or oracle.

Architecture



Proof so far

20

Solidity tests

unit · fuzz · invariant

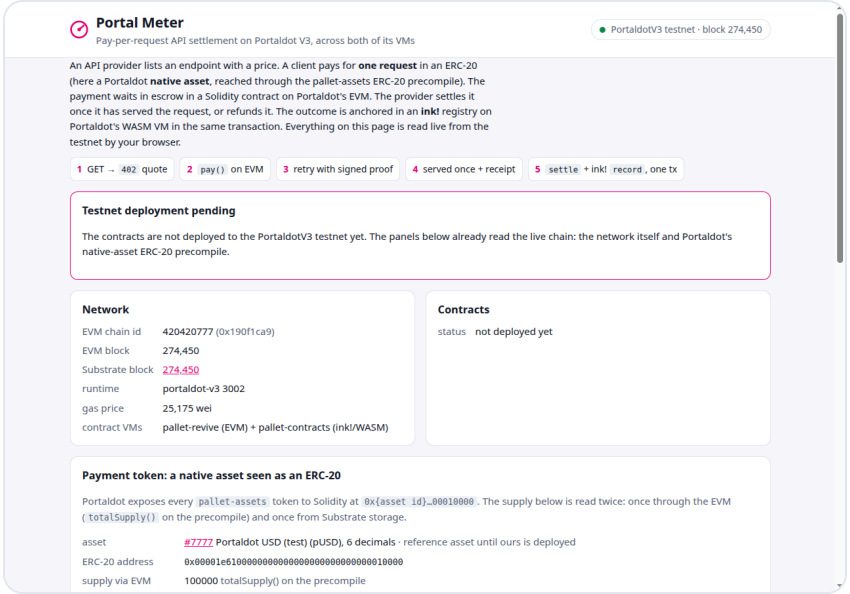
8

ink! unit tests

25

Node tests incl.

end to end on a chain



- CI on every push: Foundry, ink! release build (cargo-contract 5.0.3), Node end to end
- Measured on the live testnet: the ink! code is accepted by an instantiate dry-run; Ethereum transactions to the runtime pallets address dispatch `batchAll(revive.call, contracts.call)`
- Live page reads both VMs from the browser and verifies receipts
- Open source, MIT: github.com/swiftbotlab/portal-meter

Testnet deployment

PLACEHOLDER: filled after the testnet deployment

MeterSettlement address and deploy tx · MeterRegistry (ink!) address · native asset id · endpoint registration tx (Solidity + ink! in one Ethereum tx) · one served request (pay, settle + anchor) · one refunded request (pay, refund + anchor) · buyer feedback tx

The deployment runs with one command (scripts/deploy-testnet.mjs) once the deployer account holds testnet tPOT.

Who pays, and how it grows

- **Users:** data/API providers on Portaldot (indexers, RPC and analytics services, AI tool endpoints) and the apps and agents that call them.
- **First product:** the paid Portaldot data API in this repo (cross-VM account and asset views): a real endpoint other builders can call per request.
- **Revenue (planned):** a small protocol fee on settlement (basis points, per deployment; not in the testnet contract yet) and a hosted seller for providers who do not want to run one.
- **Growth:** drop-in middleware for Express/Fastify, listing in the Portaldot ecosystem directory, wallet support for 402 quotes.

Roadmap, and why keep supporting it

Next 4 weeks

npm middleware package · batch settlement (many requests per transaction) · provider bond held in the ink! registry, slashable on disputes · mainnet migration review

Next quarter

prepaid credit channels for high-frequency callers · explorer / wallet integration of receipts · more paid Portaldot data endpoints

What Portaldot gets

open-source middleware every API builder can reuse · real fee-paying transactions on both VMs · a reference for cross-VM design (documented, tested, measured)

Team

Swift Bot Labs, a small studio building payment and API infrastructure. Builds on our MIT-licensed HSK Meter (Oct 2026); this repository was written for Portaldot during the Hacker House.